Résolution du Problème du Voyageur de Commerce par Métaheuristiques et Approche Hybride

Réalisé par : Loubna Elasri

Année universitaire : 2025–2026

Résumé

Le problème du voyageur de commerce (Traveling Salesman Problem, TSP) constitue l'un des problèmes les plus étudiés en optimisation combinatoire. Il consiste à déterminer une tournée de coût minimal permettant de visiter un ensemble de villes exactement une fois avant de revenir au point de départ.

En raison de la complexité combinatoire du problème, les méthodes exactes deviennent rapidement inefficaces lorsque la taille des instances augmente. Les métaheuristiques représentent donc une alternative efficace permettant d'obtenir des solutions de bonne qualité dans des temps de calcul raisonnables.

Dans ce projet, plusieurs approches sont étudiées pour résoudre le TSP : une heuristique gloutonne basée sur la méthode du plus proche voisin, l'algorithme Threshold Accepting et le recuit simulé. Afin d'améliorer les performances obtenues, une approche hybride combinant le recuit simulé avec une recherche locale de type 2-opt est également proposée.

Les différentes méthodes sont testées sur plusieurs instances du problème et les résultats obtenus sont analysés et comparés.

1 Introduction

Le problème du voyageur de commerce (TSP) est un problème classique d'optimisation combinatoire. Il apparaît dans de nombreux domaines tels que la logistique, la planification de tournées, la robotique ou encore la bio-informatique.

Le problème consiste à déterminer la plus courte tournée permettant de visiter un ensemble de villes une seule fois chacune, avant de revenir à la ville de départ.

Soit n villes et une matrice de distances $D = (d_{ij})$ où d_{ij} représente la distance entre la ville i et la ville j.

Une solution du problème est une permutation des villes :

$$x = (x_1, x_2, \ldots, x_n)$$

Le coût d'une tournée est défini par :

$$f(x) = \sum_{k=1}^{n} d_{x_k x_{k+1}}, \quad \text{avec } x_{n+1} = x_1$$

Le TSP appartient à la classe des problèmes NP-difficiles. Cela signifie que le temps nécessaire pour trouver une solution optimale augmente de manière exponentielle avec le nombre de villes.

Pour cette raison, les heuristiques et les métaheuristiques sont largement utilisées afin d'obtenir des solutions proches de l'optimum.

Dans ce travail, plusieurs méthodes sont implémentées et comparées afin d'étudier leur efficacité pour la résolution du TSP.

2 Lecture des données

Les instances du problème sont fournies sous forme de matrices de distances stockées dans des fichiers texte. Chaque élément d_{ij} de la matrice représente la distance entre la ville i et la ville j.

Lors de la lecture des données, la matrice est stockée dans une structure de type tableau (array) permettant un accès rapide aux distances. Une vérification est également effectuée afin de garantir que $d_{ii} = 0$ pour toute ville i, ce qui correspond à l'absence de distance entre une ville et elle-même.

3 Objectif du projet

Le sujet demande d'implémenter :

— une heuristique de base du *plus proche voisin* ;
— une métaheuristique au choix pour améliorer la solution initiale ;
— une comparaison des performances en termes de coût de tournée, temps d'exécution et gain relatif.

L'approche retenue ici est un **algorithme mémétique**, c'est-à-dire une hybridation entre :

— un **algorithme génétique** pour l'exploration globale ;
— une **recherche locale 2-opt** pour l'intensification.

4 Description des données

Les expériences sont réalisées sur dix instances issues de TSPLIB :

Instance	Taille
eil51	51
berlin52	52
st70	70
pr76	76
kroA100	100
kroB100	100
rd100	100
eil101	101
lin105	105
ch130	130

Ces instances couvrent des tailles différentes, ce qui permet d'observer le comportement des deux approches sur des problèmes de plus en plus grands.

5 Heuristique gloutonne du plus proche voisin

La première méthode utilisée est l'heuristique gloutonne du plus proche voisin. Cette approche consiste à construire progressivement une tournée en choisissant à chaque étape la ville non visitée la plus proche de la ville courante.

L'algorithme fonctionne de la manière suivante :

1. Choisir aléatoirement une ville de départ.
2. Sélectionner parmi les villes non visitées celle qui minimise la distance à la ville actuelle.
3. Ajouter cette ville à la tournée.
4. Répéter l'opération jusqu'à ce que toutes les villes soient visitées.

5. Revenir à la ville de départ pour fermer la tournée.

Cette heuristique est très rapide mais elle peut conduire à des solutions de qualité limitée, car les décisions sont prises de manière locale sans considération globale.

6 Algorithme Threshold Accepting

Threshold Accepting est une méthode de recherche locale permettant d'explorer l'espace des solutions tout en acceptant certaines détériorations contrôlées.

À chaque itération, un voisin de la solution courante est généré en échangeant deux villes dans la tournée. Si la différence de coût entre la nouvelle solution et la solution courante est inférieure à un seuil S, la solution est acceptée.

Au cours de l'exécution de l'algorithme, ce seuil est progressivement diminué afin de rendre les critères d'acceptation de plus en plus stricts.

Cette stratégie permet d'éviter de rester bloqué dans des minima locaux et favorise l'exploration de nouvelles régions de l'espace de recherche.

7 Recuit simulé

Le recuit simulé est une métaheuristique inspirée du processus physique de refroidissement des matériaux.

L'algorithme accepte systématiquement les améliorations de solution. Cependant, il peut également accepter des solutions de moins bonne qualité avec une probabilité donnée par :

$$P = e^{-d/T}$$

où d représente la détérioration de la solution et T la température.

La température est progressivement réduite selon une loi de refroidissement géométrique :

$$T \leftarrow \alpha T$$

avec $0 < \alpha < 1$.

Au début de l'algorithme, la température est élevée et permet une exploration importante de l'espace de recherche. Au fur et à mesure que la température diminue, l'algorithme devient plus sélectif et se concentre sur l'amélioration de la solution.

8 Approche hybride proposée

Afin d'améliorer la qualité des solutions obtenues, une approche hybride est proposée.

Cette méthode combine le recuit simulé avec une recherche locale de type 2-opt.

L'idée principale est d'utiliser le recuit simulé pour explorer l'espace des solutions, puis d'appliquer une amélioration locale permettant d'optimiser davantage la tournée.

La méthode 2-opt consiste à supprimer deux arêtes de la tournée et à reconnecter les segments de manière différente afin de réduire la distance totale.

Cette hybridation permet généralement d'obtenir des solutions de meilleure qualité que l'utilisation d'une seule métaheuristique.

9 Structure du code source

Afin d'assurer une bonne organisation du projet, le code source a été structuré en plusieurs modules Python.

Le fichier **reader.py** permet de lire les instances du problème et de construire la matrice des distances.

Le module **tsp_utils.py** contient des fonctions utilitaires, notamment le calcul de la longueur d'une tournée et la génération de solutions voisines.

Les différents algorithmes sont implémentés dans des fichiers séparés :

— **greedy.py** : implémentation de l'heuristique du plus proche voisin.
— **threshold_accepting.py** : implémentation de l'algorithme Threshold Accepting.
— **simulated_annealing.py** : implémentation du recuit simulé.
— **hybrid_metaheuristic.py** : approche hybride combinant recuit simulé et recherche locale 2-opt.

Le script **experiments.py** permet d'exécuter les différents algorithmes sur les instances et de comparer les résultats obtenus.

Les résultats expérimentaux et les figures générées sont enregistrés dans le dossier **results**.

10 Expérimentations

Les différentes méthodes ont été testées sur plusieurs instances du problème du voyageur de commerce.

Pour chaque instance, les algorithmes suivants ont été exécutés :

— heuristique gloutonne du plus proche voisin
— Threshold Accepting
— recuit simulé
— approche hybride (recuit simulé + 2-opt)

11 Résultats expérimentaux

Afin d'évaluer les performances des différentes méthodes implémentées, plusieurs expérimentations ont été réalisées sur les instances du problème du voyageur de commerce utilisées dans ce projet. Les algorithmes testés sont : l'heuristique gloutonne du plus proche voisin, l'algorithme Threshold Accepting, le recuit simulé et l'approche hybride proposée.

Pour chaque instance, les algorithmes ont été exécutés dans les mêmes conditions expérimentales. La qualité des solutions obtenues est mesurée à l'aide de la longueur totale de la tournée produite par chaque méthode.

Les résultats obtenus sont représentés graphiquement afin de comparer les performances des différentes approches.

—	Greedy	Threshold Accepting	SA	Hybrid
Distance	2394	2220	2157	2050
Temps	0.0	2.257	2.498	4.415

Table 1 – Comparaison des différentes méthodes pour l'instance 29

—	Greedy	Threshold Accepting	SA	Hybrid
Distance	558.8489729999999	479.808103	479.808103	490.545335
Temps	0.0	2.428	3.385	5.713

Table 2 – Comparaison des différentes méthodes pour l'instance 51

L'analyse des résultats montre que l'heuristique gloutonne fournit généralement une solution initiale rapide mais de qualité limitée. En effet, cette méthode repose sur des décisions locales

qui ne garantissent pas une solution globale optimale.

Les métaheuristiques telles que Threshold Accepting et le recuit simulé permettent d'améliorer significativement la qualité des solutions obtenues. Grâce à leur capacité à explorer différentes régions de l'espace de recherche, ces méthodes sont capables d'éviter plus facilement les minima locaux.

Enfin, l'approche hybride combinant le recuit simulé avec une recherche locale de type 2-opt permet d'obtenir les meilleures performances sur les instances étudiées. Cette amélioration s'explique par la combinaison de deux mécanismes complémentaires : l'exploration globale de l'espace de recherche réalisée par le recuit simulé et l'intensification locale apportée par l'algorithme 2-opt.

Ces résultats confirment l'intérêt de l'hybridation des métaheuristiques pour résoudre des problèmes d'optimisation combinatoire complexes tels que le problème du voyageur de commerce.

12 Discussion

L'analyse des résultats met en évidence plusieurs observations importantes.

Tout d'abord, l'heuristique gloutonne est extrêmement rapide mais fournit souvent des solutions de qualité inférieure.

Les métaheuristiques telles que Threshold Accepting et le recuit simulé permettent d'améliorer significativement les résultats grâce à leur capacité d'exploration.

Enfin, l'approche hybride combinant recuit simulé et recherche locale offre les meilleures performances, car elle combine exploration globale et amélioration locale.

13 Conclusion

Dans ce projet, plusieurs méthodes heuristiques et métaheuristiques ont été étudiées afin de résoudre le problème du voyageur de commerce (TSP). Dans un premier temps, une heuristique gloutonne basée sur la stratégie du plus proche voisin a été implémentée afin de construire rapidement une solution initiale. Cette approche présente l'avantage d'être simple et rapide à mettre en œuvre, mais les résultats montrent que la qualité des solutions obtenues reste limitée en raison du caractère local des décisions prises.

Afin d'améliorer ces solutions initiales, deux métaheuristiques à solution unique ont été mises en œuvre : l'algorithme Threshold Accepting et le recuit simulé. Ces méthodes permettent d'explorer plus efficacement l'espace des solutions en acceptant temporairement certaines détériorations, ce qui permet d'éviter le blocage dans des minima locaux et d'obtenir des solutions de meilleure qualité.

Une approche hybride combinant le recuit simulé et la recherche locale 2-opt a été proposée. Les résultats montrent qu'elle permet d'obtenir de meilleures solutions en combinant exploration globale et optimisation locale.

Les expérimentations réalisées confirment l'efficacité des métaheuristiques pour résoudre des problèmes d'optimisation combinatoire complexes comme le TSP. Comme perspective future, il serait intéressant d'étudier d'autres approches et de tester leurs performances sur des instances de plus grande taille.